

Kubernetes, Cloud & Observability – Complete Guide

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

 **Study window:** 18 Jan 2027 – 28 Jan 2027 (Days 155–165)

 **Your slots (IST):** Mon–Fri **7:30–8:30pm & 9:00–10:00pm** · Sat–Sun **9:00am–12:00pm, 3:00–5:30pm, 6:00–8:30pm**

 **Calendar reminders:** harsh.gupta@getreelax.com – import [calendar/mern-mastery-study.ics](https://calendar.google.com/calendar/ical/harsh.gupta%40getreelax.com/public/calendar/mern-mastery-study.ics)

Kubernetes internals, Docker optimization, AWS advanced services, observability stack, SLO/SLA, and 80 LPA interview preparation.

Target Level: Google L5, Meta E4, Amazon SDE3, Microsoft L63+ (80+ LPA TC in India)

PREREQUISITES – Complete FIRST

Order	PDF
1	07-auth-devops-complete.pdf §Docker (used in Month 4 projects)
2	10c-system-design-prerequisites-complete.pdf §Monitoring
3	12-microservices-kafka-complete.pdf (optional but recommended)

When: Month 6, Days 161–165.

Table of Contents

- [1. Kubernetes Core Concepts](#)
- [2. Kubernetes Workloads & Networking](#)
- [3. Kubernetes Scaling & Configuration](#)
- [4. Docker Deep Dive](#)
- [5. AWS Advanced Services](#)
- [6. Observability Stack](#)
- [7. SLI, SLO, SLA & Error Budgets](#)
- [8. Interview Q&A \(15 Questions\)](#)
- [9. Where to Practice](#)

1. Kubernetes Core Concepts

Architecture Overview

Control Plane

- API Server ↔ etcd (state store)
- Scheduler (assigns pods to nodes)
- Controller Manager (reconciliation loops)
- Cloud Controller Manager (cloud integration)

↑ API

Worker Node

- kubelet (pod lifecycle)
- kube-proxy (network rules)
- Container Runtime (containerd/CRI-O)

Pod 1

Pod 2

Pod 3

Pods

Smallest deployable unit. One or more containers sharing network namespace and storage.

```
apiVersion: v1
kind: Pod
metadata:
  name: api-server
  labels:
    app: api
    tier: backend
spec:
  containers:
    - name: api
      image: myapp:v1.2.3
      ports:
        - containerPort: 3000
      resources:
        requests:
          cpu: "250m"          # 0.25 CPU cores
          memory: "256Mi"
        limits:
          cpu: "500m"
          memory: "512Mi"
      livenessProbe:
        httpGet:
          path: /health
          port: 3000
        initialDelaySeconds: 10
        periodSeconds: 15
      readinessProbe:
        httpGet:
          path: /ready
          port: 3000
        initialDelaySeconds: 5
        periodSeconds: 5
      env:
        - name: DATABASE_URL
          valueFrom:
            secretKeyRef:
              name: db-credentials
              key: url
  restartPolicy: Always
```

Multi-container pod patterns:

- **Sidecar:** Helper container (logging agent, proxy)
- **Ambassador:** Proxy to external service
- **Adapter:** Transform output format

Pod lifecycle phases: Pending → Running → Succeeded/Failed. Containers: Waiting → Running → Terminated.

Deployments

Declarative updates for ReplicaSets and Pods.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: api-deployment
spec:
  replicas: 3
  selector:
    matchLabels:
      app: api
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1      # Extra pods during update
      maxUnavailable: 0 # Zero downtime
  template:
    metadata:
      labels:
        app: api
    spec:
      containers:
        - name: api
          image: myapp:v1.2.3
          ports:
            - containerPort: 3000
```

Deployment strategies:

Strategy	Description	Use Case
Rolling Update	Gradually replace old pods	Default, zero-downtime
Recreate	Kill all, then create new	Dev/staging, stateful changes
Blue/Green	Two full environments, switch traffic	Instant rollback needed
Canary	Route small % to new version	Risk mitigation

Rollout commands:

```
kubectl rollout status deployment/api-deployment  
kubectl rollout history deployment/api-deployment  
kubectl rollout undo deployment/api-deployment          # Rollback  
kubectl rollout undo deployment/api-deployment --to-revision=2
```

2. Kubernetes Workloads & Networking

Services

Stable network endpoint for a set of pods.

```
# ClusterIP (internal only)
apiVersion: v1
kind: Service
metadata:
  name: api-service
spec:
  selector:
    app: api
  ports:
    - port: 80
      targetPort: 3000
  type: ClusterIP

---

# LoadBalancer (external access via cloud LB)
apiVersion: v1
kind: Service
metadata:
  name: api-external
spec:
  selector:
    app: api
  ports:
    - port: 443
      targetPort: 3000
  type: LoadBalancer

---

# Headless (direct pod DNS, for StatefulSets)
apiVersion: v1
kind: Service
metadata:
  name: db-headless
spec:
  clusterIP: None
  selector:
    app: postgres
  ports:
    - port: 5432
```

Service types:

Type	Access	Use Case
ClusterIP	Internal only	Service-to-service communication
NodePort	External via node IP:port	Development, bare metal
LoadBalancer	External via cloud LB	Production external access
ExternalName	DNS CNAME	External service proxy

DNS resolution: `api-service.default.svc.cluster.local` → ClusterIP → kube-proxy → Pod IPs

Ingress

HTTP/HTTPS routing to services. Layer 7 load balancing.

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
  annotations:
    nginx.ingress.kubernetes.io/rate-limit: "100"
    cert-manager.io/cluster-issuer: "letsencrypt-prod"
spec:
  ingressClassName: nginx
  tls:
    - hosts:
        - api.myapp.com
      secretName: api-tls
  rules:
    - host: api.myapp.com
      http:
        paths:
          - path: /api/v1
            pathType: Prefix
            backend:
              service:
                name: api-service
                port:
                  number: 80
          - path: /ws
            pathType: Prefix
            backend:
              service:
                name: websocket-service
                port:
                  number: 80
    - host: admin.myapp.com
      http:
        paths:
          - path: /
            pathType: Prefix
            backend:
              service:
                name: admin-service
                port:
                  number: 80
```

Ingress controllers: NGINX Ingress, AWS ALB Ingress Controller, Traefik, Istio Gateway

Ingress vs LoadBalancer Service:

- Ingress: One LB for many services (cost-effective), path/host routing, TLS termination
- LoadBalancer: One LB per service (expensive), Layer 4

StatefulSets

For stateful applications requiring stable identity and storage.

```
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: postgres
spec:
  serviceName: postgres-headless
  replicas: 3
  selector:
    matchLabels:
      app: postgres
  template:
    metadata:
      labels:
        app: postgres
    spec:
      containers:
        - name: postgres
          image: postgres:15
          volumeMounts:
            - name: data
              mountPath: /var/lib/postgresql/data
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 10Gi
```

Guarantees: Stable pod names (postgres-0, postgres-1), stable storage (PVC per pod), ordered deployment/scaling.

3. Kubernetes Scaling & Configuration

Horizontal Pod Autoscaler (HPA)

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: api-hpa
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: api-deployment
  minReplicas: 2
  maxReplicas: 20
  metrics:
    - type: Resource
      resource:
        name: cpu
        target:
          type: Utilization
          averageUtilization: 70
    - type: Resource
      resource:
        name: memory
        target:
          type: Utilization
          averageUtilization: 80
    - type: Pods
      pods:
        metric:
          name: http_requests_per_second
        target:
          type: AverageValue
          averageValue: "1000"
  behavior:
    scaleUp:
      stabilizationWindowSeconds: 60
      policies:
        - type: Percent
          value: 100
          periodSeconds: 60
    scaleDown:
      stabilizationWindowSeconds: 300
      policies:
        - type: Percent
```

```
value: 10
periodSeconds: 60
```

HPA algorithm: $\text{desiredReplicas} = \text{ceil}(\text{currentReplicas} \times (\text{currentMetric} / \text{targetMetric}))$

Custom metrics: Requires metrics-server (CPU/memory) or Prometheus adapter (custom metrics like queue depth, request rate).

Vertical Pod Autoscaler (VPA)

Automatically adjusts CPU/memory requests and limits based on historical usage. Useful for right-sizing workloads that HPA can't handle (stateful, non-horizontally-scalable).

Cluster Autoscaler

Adds/removes nodes based on pending pods. Works with cloud provider auto-scaling groups (AWS ASG, GCE MIG).

```
Pending pods → Cluster Autoscaler → Add node to ASG → Pod scheduled
Low utilization → Cluster Autoscaler → Drain node → Remove from ASG
```

ConfigMaps

Non-sensitive configuration data.

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: app-config
data:
  LOG_LEVEL: "info"
  MAX_CONNECTIONS: "100"
  app.properties: |
    feature.newUI=true
    cache.ttl=3600

---
# Usage in Pod
spec:
  containers:
    - name: api
      envFrom:
        - configMapRef:
            name: app-config
      volumeMounts:
        - name: config-volume
          mountPath: /etc/config
  volumes:
    - name: config-volume
      configMap:
        name: app-config
```

Secrets

Sensitive data (base64 encoded, NOT encrypted by default).

```
apiVersion: v1
kind: Secret
metadata:
  name: db-credentials
type: Opaque
data:
  username: cG9zdGdyZXM= # base64("postgres")
  password: cGFzc3dvcmQxMjM= # base64("password123")
  url: cG9zdGdyZXM6Ly8uLi4= # base64 connection string

---
# Better: External Secrets Operator
# Syncs from AWS Secrets Manager / HashiCorp Vault
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata:
  name: db-credentials
spec:
  refreshInterval: 1h
  secretStoreRef:
    name: aws-secrets-manager
    kind: ClusterSecretStore
  target:
    name: db-credentials
  data:
    - secretKey: password
      remoteRef:
        key: prod/database/password
```

Secret best practices:

- Never commit secrets to git (use Sealed Secrets or External Secrets Operator)
- Enable encryption at rest (etcd encryption provider)
- Use RBAC to restrict secret access
- Rotate secrets regularly

4. Docker Deep Dive

Multi-Stage Builds

Problem: Build tools (compilers, dev dependencies) bloat production images.

```
# Stage 1: Build
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production=false
COPY . .
RUN npm run build

# Stage 2: Production
FROM node:20-alpine AS production
WORKDIR /app

# Security: non-root user
RUN addgroup -g 1001 -S nodejs && adduser -S nodejs -u 1001

COPY --from=builder /app/dist ./dist
COPY --from=builder /app/node_modules ./node_modules
COPY --from=builder /app/package.json ./

USER nodejs
EXPOSE 3000

HEALTHCHECK --interval=30s --timeout=3s --start-period=5s --retries=3 \
  CMD wget --no-verbose --tries=1 --spider http://localhost:3000/health || exit 1

CMD ["node", "dist/server.js"]
```

Result: Build stage ~1.2GB → Production stage ~150MB

Image Optimization Techniques

Technique	Impact	Example
Multi-stage builds	60-80% size reduction	Builder + runtime stages
Alpine base images	50-70% smaller	<code>node:20-alpine</code> vs <code>node:20</code>
.dockerignore	Faster builds, smaller context	Exclude node_modules, .git
Layer caching	Faster rebuilds	COPY package.json before source
Distroless images	Minimal attack surface	<code>gcr.io/distroless/nodejs20</code>
Combine RUN commands	Fewer layers	<code>RUN apt-get update && apt-get install -y ... && rm -rf /var/lib/apt/lists/*</code>

Optimal layer ordering:

```
# 1. Base image (rarely changes)
FROM node:20-alpine

# 2. Dependencies (changes occasionally)
COPY package*.json ./
RUN npm ci --only=production

# 3. Source code (changes frequently)
COPY . .

# 4. Build (if needed)
RUN npm run build
```

Docker Security

```
# Non-root user
RUN adduser -D appuser
USER appuser

# Read-only filesystem
docker run --read-only --tmpfs /tmp myapp

# No new privileges
securityContext:
  runAsNonRoot: true
  readOnlyRootFilesystem: true
  allowPrivilegeEscalation: false
  capabilities:
    drop: ["ALL"]
```

Docker Compose for Local Development

```
version: '3.8'
services:
  api:
    build:
      context: .
      target: builder # Use builder stage for dev (includes dev deps)
    ports:
      - "3000:3000"
    volumes:
      - ./app
      - /app/node_modules
    environment:
      - DATABASE_URL=postgres://postgres:password@db:5432/myapp
      - REDIS_URL=redis://redis:6379
    depends_on:
      db:
        condition: service_healthy

  db:
    image: postgres:15-alpine
    environment:
      POSTGRES_PASSWORD: password
      POSTGRES_DB: myapp
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s
      timeout: 3s
      retries: 5

  redis:
    image: redis:7-alpine
    ports:
      - "6379:6379"

volumes:
  pgdata:
```

5. AWS Advanced Services

EC2 (Elastic Compute Cloud)

Instance types for different workloads:

Family	Use Case	Example
t3/t4g	Burstable, general purpose	Dev, low-traffic web
m6i/m7g	Balanced compute/memory	Application servers
c6i/c7g	Compute optimized	Batch processing, gaming
r6i/r7g	Memory optimized	Caching, in-memory DB
p4/p5	GPU instances	ML training/inference

Auto Scaling Group (ASG):

CloudWatch Alarm (CPU > 70%)

- ASG Scale Out (add instances)
- Load Balancer registers new instance
- Traffic distributed

CloudWatch Alarm (CPU < 30% for 10 min)

- ASG Scale In (remove instance)
- Connection draining → Terminate

Spot Instances: Up to 90% discount, can be interrupted with 2-min notice. Use for fault-tolerant, stateless workloads (batch processing, CI/CD runners, Kubernetes node groups).

S3 (Simple Storage Service)

Storage classes:

Class	Use Case	Cost
S3 Standard	Frequently accessed	\$\$\$
S3 Intelligent-Tiering	Unknown/changing access	\$\$
S3 Standard-IA	Infrequent access	\$\$
S3 Glacier Instant	Archive, instant retrieval	\$
S3 Glacier Deep Archive	Long-term archive (12hr retrieval)	¢

Key patterns:

```
// Pre-signed URL for direct upload (bypass backend)
const url = await s3.getSignedUrl('putObject', {
  Bucket: 'my-bucket',
  Key: `uploads/${userId}/${filename}`,
  Expires: 3600,
  ContentType: 'image/jpeg',
});

// Multipart upload for large files (> 100MB)
// S3 Event Notification → Lambda (image processing)
// S3 → CloudFront (CDN caching)
```

S3 consistency: Read-after-write consistency for new objects. Eventual consistency for overwrites and deletes (now strong consistency for all operations since Dec 2020).

RDS (Relational Database Service)

Engine options: PostgreSQL, MySQL, Aurora (AWS-proprietary, 3x performance)

High availability:

```
Primary (AZ-a) ← synchronous replication → Standby (AZ-b)
    |
    ├── Automated backups to S3
    └── Read replicas (async, up to 15) →
```

Key features:

- Multi-AZ: Automatic failover (~60s), synchronous replication
- Read replicas: Scale reads, cross-region disaster recovery

- Automated backups: Point-in-time recovery (5-min granularity)
- Parameter groups: Engine configuration tuning

Aurora specifics:

- Storage auto-scales (10GB → 128TB)
- 6 copies across 3 AZs
- Aurora Serverless v2: Auto-scaling capacity (0.5 - 128 ACUs)
- Global Database: Cross-region replication (< 1s lag)

ElastiCache

In-memory caching: Redis or Memcached.

```
Application → Check ElastiCache → Hit: return cached data
                                     → Miss: query RDS → store in cache → return
```

Redis use cases:

- Session store
- Rate limiting (sliding window counters)
- Leaderboards (sorted sets)
- Pub/sub for real-time features
- Distributed locks (Redlock)

Configuration:

```
Node type: cache.r6g.large (13GB memory)
Cluster mode: Enabled (sharding across nodes)
Replicas: 2 per shard (Multi-AZ failover)
Eviction policy: allkeys-lru (evict least recently used when full)
```

Lambda (Serverless)

Event-driven compute:

```
// API Gateway → Lambda
export const handler = async (event) => {
  const { userId } = event.pathParameters;
  const user = await getUserFromDynamoDB(userId);
  return {
    statusCode: 200,
    headers: { 'Content-Type': 'application/json' },
    body: JSON.stringify(user),
  };
};

// S3 Event → Lambda (image resize)
// SQS → Lambda (async processing)
// EventBridge → Lambda (scheduled tasks)
// DynamoDB Streams → Lambda (CDC)
```

Limits and considerations:

Aspect	Limit	Mitigation
Timeout	15 minutes	Step Functions for longer workflows
Memory	128MB - 10GB	More memory = more CPU
Payload	6MB sync, 256KB async	S3 for large payloads
Concurrency	1000 default (soft)	Request limit increase
Cold start	100ms-2s	Provisioned concurrency, ARM (Graviton)

Lambda best practices:

- Keep functions small and focused (single responsibility)
- Use environment variables for config, Secrets Manager for secrets
- ARM64 (Graviton2) for 20% cost savings
- Provisioned concurrency for latency-sensitive paths
- Dead letter queue (DLQ) for failed invocations

SQS (Simple Queue Service)

Queue types:

Type	Ordering	Throughput	Use Case
Standard	Best-effort	Unlimited	General async processing
FIFO	Guaranteed	300 msg/sec	Order processing, financial

Pattern: SQS + Lambda:

```
API → SQS (buffer) → Lambda (auto-scales with queue depth)
                        → DLQ (failed messages after 3 retries)
```

Visibility timeout: After consumer receives message, it's hidden for N seconds. If not deleted within timeout, message reappears (retry). Set timeout > max processing time.

CloudFront (CDN)

```
User (Mumbai) → CloudFront Edge (Mumbai) → Cache HIT → Fast response
                                                → Cache MISS → Origin (S3/ALB in us-east-1)
```

Features:

- Edge locations worldwide (400+)
- SSL/TLS termination
- Custom cache policies (TTL, query string handling)
- Origin Access Identity (OAI) for private S3 buckets
- Lambda@Edge for request/response manipulation
- WAF integration for security

IAM (Identity and Access Management)

Core concepts:

IAM User → Access Key (programmatic) + Password (console)
IAM Role → Assumed by services/users (no long-term credentials)
IAM Policy → JSON document defining permissions

```
{
  "Version": "2012-10-17",
  "Statement": [{
    "Effect": "Allow",
    "Action": ["s3:GetObject", "s3:PutObject"],
    "Resource": "arn:aws:s3:::my-bucket/uploads/*",
    "Condition": {
      "StringEquals": { "aws:PrincipalTag/team": "backend" }
    }
  }]
}
```

Best practices:

- Never use root account for daily operations
- Use roles for EC2/Lambda (not access keys on instances)
- Principle of least privilege
- Enable MFA for all users
- Use IAM Access Analyzer to find overly permissive policies
- Rotate access keys regularly

6. Observability Stack

Three Pillars of Observability

Metrics (What?)	Logs (Why?)	Traces (Where?)
Prometheus Grafana	ELK/Loki CloudWatch	Jaeger/ Zipkin/ Tempo

Prometheus & Grafana

Prometheus: Pull-based metrics collection and alerting.

```
// Instrument Node.js app with prom-client
const promClient = require('prom-client');

const httpRequestDuration = new promClient.Histogram({
  name: 'http_request_duration_seconds',
  help: 'Duration of HTTP requests in seconds',
  labelNames: ['method', 'route', 'status_code'],
  buckets: [0.01, 0.05, 0.1, 0.5, 1, 2, 5],
});

const httpRequestTotal = new promClient.Counter({
  name: 'http_requests_total',
  help: 'Total number of HTTP requests',
  labelNames: ['method', 'route', 'status_code'],
});

// Middleware
app.use((req, res, next) => {
  const end = httpRequestDuration.startTimer({ method: req.method, route: req.route });
  res.on('finish', () => {
    end({ status_code: res.statusCode });
    httpRequestTotal.inc({ method: req.method, route: req.route?.path, status_code: res.statusCode });
  });
  next();
});

// Metrics endpoint
app.get('/metrics', async (req, res) => {
  res.set('Content-Type', promClient.register.contentType);
  res.end(await promClient.register.metrics());
});
```

Key metric types:

- **Counter:** Monotonically increasing (total requests, errors)
- **Gauge:** Can go up/down (active connections, queue depth, memory usage)
- **Histogram:** Distribution of values (request latency buckets)
- **Summary:** Similar to histogram with quantiles

PromQL examples:

```
# Request rate (requests per second)
rate(http_requests_total[5m])

# p99 latency
histogram_quantile(0.99, rate(http_request_duration_seconds_bucket[5m]))

# Error rate percentage
sum(rate(http_requests_total{status_code=~"5.."}[5m])) / sum(rate(http_requests_to

# Alert: error rate > 1% for 5 minutes
ALERT HighErrorRate
  IF sum(rate(http_requests_total{status_code=~"5.."}[5m])) / sum(rate(http_reques
  FOR 5m
  LABELS { severity = "critical" }
  ANNOTATIONS { summary = "Error rate above 1%" }
```

Grafana: Visualization and dashboarding. Connect to Prometheus, CloudWatch, Elasticsearch data sources. Create dashboards for RED method (Rate, Errors, Duration).

Distributed Tracing (Jaeger)

Problem: Request spans 5 microservices. Which one is slow?

```
Trace ID: abc123
├─ Span: API Gateway (50ms)
│   └─ Span: Auth Service (10ms)
│       └─ Span: Order Service (200ms)
│           └─ Span: DB Query (150ms) ← BOTTLENECK
│               └─ Span: Cache Check (2ms)
└─ Span: Payment Service (100ms)
    └─ Span: Stripe API (95ms)
Total: 350ms
```

OpenTelemetry instrumentation:

```
const { NodeTracerProvider } = require('@opentelemetry/sdk-trace-node');
const { JaegerExporter } = require('@opentelemetry/exporter-jaeger');
const { registerInstrumentations } = require('@opentelemetry/instrumentation');
const { HttpInstrumentation } = require('@opentelemetry/instrumentation-http');
const { ExpressInstrumentation } = require('@opentelemetry/instrumentation-express');

const provider = new NodeTracerProvider();
provider.addSpanProcessor(new BatchSpanProcessor(new JaegerExporter({
  endpoint: 'http://jaeger:14268/api/traces',
})));
provider.register();

registerInstrumentations({
  instrumentations: [
    new HttpInstrumentation(),
    new ExpressInstrumentation(),
  ],
});

// Custom span
const tracer = trace.getTracer('order-service');
const span = tracer.startSpan('processOrder');
try {
  await processOrder(order);
  span.setStatus({ code: SpanStatusCode.OK });
} catch (error) {
  span.setStatus({ code: SpanStatusCode.ERROR, message: error.message });
  throw error;
} finally {
  span.end();
}
```

Trace context propagation: W3C Trace Context headers (`traceparent` , `tracestate`) passed between services via HTTP headers or Kafka message headers.

Structured Logging

Never log unstructured strings in production:

```
// ❌ Bad
console.log(`User ${userId} placed order ${orderId} for $${total}`);

// ✅ Good – structured JSON logging
const logger = require('pino')({
  level: process.env.LOG_LEVEL || 'info',
  formatters: {
    level: (label) => ({ level: label }),
  },
  base: { service: 'order-service', version: '1.2.3' },
});

logger.info({
  event: 'order.created',
  userId,
  orderId,
  total,
  itemCount: items.length,
  traceId: span.spanContext().traceId,
}, 'Order created successfully');
```

Output:

```
{
  "level": "info",
  "time": 1690896000000,
  "service": "order-service",
  "version": "1.2.3",
  "event": "order.created",
  "userId": "user_123",
  "orderId": "ord_456",
  "total": 99.99,
  "itemCount": 3,
  "traceId": "abc123def456",
  "msg": "Order created successfully"
}
```

Log aggregation stack:

- **ELK:** Elasticsearch + Logstash + Kibana
- **Loki:** Grafana's log aggregation (label-based, cheaper than full-text indexing)
- **CloudWatch Logs Insights:** AWS-native querying

Correlation: Include `traceId`, `spanId`, `requestId` in every log entry to correlate logs with traces.

7. SLI, SLO, SLA & Error Budgets

Definitions

Term	Definition	Example
SLI (Indicator)	Measurable aspect of service level	Request latency p99, error rate, availability
SLO (Objective)	Target value for an SLI	p99 latency < 200ms, availability 99.9%
SLA (Agreement)	Contract with consequences	99.9% uptime or credits refunded
Error Budget	Allowed unreliability = 100% - SLO	99.9% SLO → 0.1% error budget = 43.8 min/month

SLI Selection

For a user-facing API:

SLI	Measurement	Good SLO
Availability	Successful requests / Total requests	99.9% (three nines)
Latency	p99 response time	< 300ms
Correctness	Correct responses / Total responses	99.99%
Freshness	Data age for async pipelines	< 5 minutes

"Good" events definition (Google SRE):

- Availability: HTTP 200-399 responses (not 500s, not timeouts)
- Latency: Requests completed in < 300ms (for latency SLO)

Error Budget Policy

SL0: 99.9% availability (error budget = 43.8 min/month)

Budget remaining > 50%: Normal development, feature work encouraged

Budget remaining 25–50%: Caution – prioritize reliability work

Budget remaining < 25%: Feature freeze – focus on reliability

Budget exhausted (0%): All hands on reliability – no new features until budget reset

Error budget in practice:

```
# Error budget remaining (30-day window)
1 - (
  sum(increase(http_requests_total{status_code=~"5.."}[30d]))
  / sum(increase(http_requests_total[30d]))
) / (1 - 0.999) # 0.999 = 99.9% SLO

# Burn rate alert (how fast we're consuming error budget)
# 14.4x burn rate = budget exhausted in 2 hours
sum(rate(http_requests_total{status_code=~"5.."}[1h]))
/ sum(rate(http_requests_total[1h]))
> 14.4 * 0.001 # 14.4x the allowed error rate
```

Multi-Window Burn Rate Alerts

Google SRE recommends alerting on burn rate, not absolute error rate:

Alert	Window	Burn Rate	Time to Exhaust
Page (critical)	1 hour	14.4x	2 hours
Page (critical)	5 min	14.4x	2 hours
Ticket (warning)	6 hours	6x	5 days
Ticket (warning)	30 days	1x	30 days

8. Interview Q&A (15 Questions)

Q1: Explain the difference between a Pod, Deployment, and Service in Kubernetes.

Answer: These three resources work together but serve distinct purposes in the Kubernetes architecture.

A **Pod** is the smallest deployable unit — one or more containers sharing network namespace (same IP), storage volumes, and lifecycle. Pods are ephemeral; when a pod dies, it's gone forever with a new IP. You rarely create pods directly in production.

A **Deployment** provides declarative management for pods. You specify desired state (3 replicas, image v1.2.3), and the Deployment controller creates a ReplicaSet that ensures the correct number of pod replicas are running. Deployments enable rolling updates (zero-downtime deployments), rollbacks, and scaling. When you update the image, the Deployment gradually replaces old pods with new ones.

A **Service** provides a stable network endpoint for a dynamic set of pods. Since pod IPs change on restart, Services create a persistent ClusterIP (or LoadBalancer/NodePort) with DNS name (`api-service.default.svc.cluster.local`). kube-proxy maintains iptables/IPVS rules routing Service traffic to healthy pod endpoints. Services enable service discovery and load balancing.

Together: Deployment manages pod lifecycle → Service provides stable networking → Pods run your application containers.

Q2: How does Kubernetes HPA work and what metrics can it use?

Answer: Horizontal Pod Autoscaler automatically scales the number of pod replicas based on observed metrics, ensuring applications handle varying load without manual intervention.

The HPA controller periodically (every 15 seconds by default) queries the metrics API for current resource utilization. It calculates desired replicas using: $\text{desiredReplicas} = \text{ceil}(\text{currentReplicas} \times (\text{currentMetricValue} / \text{targetMetricValue}))$. For example, with 4 replicas at 80% CPU and target 50%: $\text{ceil}(4 \times (80/50)) = \text{ceil}(6.4) = 7$ replicas.

Supported metrics:

1. **Resource metrics** (built-in via metrics-server): CPU utilization, memory utilization

2. **Custom metrics** (via Prometheus adapter): HTTP requests per second, queue depth, active connections
3. **External metrics** (via cloud provider adapter): SQS queue length, CloudWatch metrics

Configuration best practices:

- Set `minReplicas` ≥ 2 for high availability
- Configure scale-down stabilization window (default 300s) to prevent flapping
- Set appropriate resource requests (HPA compares usage against requests, not limits)
- Use custom metrics for application-specific scaling (e.g., Kafka consumer lag)

Limitations: HPA scales pods, not nodes — you need Cluster Autoscaler for node-level scaling. HPA won't help if pods can't scale horizontally (StatefulSets with local state). VPA handles vertical scaling (adjusting CPU/memory per pod) as a complementary tool.

Q3: What is the difference between liveness and readiness probes?

Answer: Both are health checks but serve fundamentally different purposes in Kubernetes pod lifecycle management.

Liveness probe determines if the container is alive and functioning. If it fails, kubelet kills the container and restarts it according to the restart policy. Use liveness probes to detect deadlocks, infinite loops, or states where the process is running but not functioning. Example: an HTTP endpoint `/health` that checks if the event loop is responsive. Configure with care — too aggressive liveness probes cause restart loops (CrashLoopBackOff). Set `initialDelaySeconds` high enough for the app to start, and `failureThreshold` to tolerate transient issues.

Readiness probe determines if the container is ready to receive traffic. If it fails, the pod's IP is removed from Service endpoints — no traffic is routed to it, but the container is NOT restarted. Use readiness probes for: application startup (loading caches, connecting to DB), temporary overload (circuit breaker open), dependency failures (database temporarily unavailable). When readiness fails, Kubernetes waits for it to pass again before routing traffic.

Example scenario: App starts, connects to database. Readiness probe checks DB connection — fails until DB is ready (pod not in service endpoints). Once DB connects, readiness passes (traffic flows). Later, DB connection pool exhausted — readiness fails (traffic stops), app recovers, readiness passes again. Liveness never failed — no unnecessary restarts.

Startup probe (K8s 1.16+): For slow-starting containers. Disables liveness/readiness until startup probe succeeds. Prevents liveness from killing containers that are still initializing.

Q4: Explain Docker multi-stage builds and image optimization.

Answer: Multi-stage builds use multiple `FROM` statements in a single Dockerfile, each representing a build stage. Only the final stage becomes the image, but earlier stages can copy artifacts into it.

The primary benefit is separating build-time dependencies from runtime. A Node.js app might need TypeScript compiler, webpack, and devDependencies to build (1.2GB), but production only needs compiled JavaScript and production node_modules (150MB). Stage 1 installs everything and builds; Stage 2 copies only the compiled output and production dependencies.

Optimization techniques beyond multi-stage:

1. **Alpine base images:** `node:20-alpine` (180MB) vs `node:20` (1GB). Use distroless (`gcr.io/distroless/nodejs20`) for minimal attack surface.
2. **Layer caching:** Order Dockerfile commands from least to most frequently changing. Copy `package.json` and run `npm ci` before copying source code — dependency layer is cached until package.json changes.
3. **.dockerignore:** Exclude `node_modules`, `.git`, test files from build context. Faster builds, smaller context.
4. **Combine RUN commands:** Each RUN creates a layer. Chain commands with `&&` and clean up in the same layer: `RUN apt-get update && apt-get install -y curl && rm -rf /var/lib/apt/lists/*`
5. **Non-root user:** Security best practice. Create user in Dockerfile, switch with `USER nodejs`.
6. **Specific tags:** Use `node:20.11.0-alpine3.19`, not `node:latest` (reproducible builds).

For 80 LPA interviews, quantify the impact: "Multi-stage reduced our image from 1.2GB to 150MB, cutting deployment time from 3 minutes to 30 seconds and reducing attack surface by eliminating build tools from production."

Q5: Compare RDS Multi-AZ vs Read Replicas.

Answer: Both provide data redundancy but serve different purposes.

Multi-AZ deployment maintains a synchronous standby replica in a different Availability Zone. Every write to the primary is synchronously replicated to the standby before acknowledgment. If the primary fails, AWS automatically fails over to the standby (typically 60-120 seconds). The standby is

NOT accessible for reads — it's purely for disaster recovery. Use Multi-AZ for production databases where high availability is critical. It doubles your cost (you pay for both instances).

Read Replicas are asynchronous copies of the primary, accessible for read queries. You can create up to 15 read replicas (5 for SQL Server). Replication lag is typically milliseconds to seconds. Use read replicas to: scale read-heavy workloads (offload SELECT queries from primary), serve analytics/reporting queries without impacting production, and provide cross-region disaster recovery (promote replica to primary in another region).

Key differences:

Aspect	Multi-AZ	Read Replica
Purpose	High availability	Read scaling
Replication	Synchronous	Asynchronous
Accessible for reads	No	Yes
Failover	Automatic	Manual promotion
Lag	Zero (sync)	Milliseconds to seconds
Count	1 standby	Up to 15

Production pattern: Use both. Multi-AZ for automatic failover on primary failure. Read replicas for scaling reads. Application routes writes to primary endpoint, reads to replica endpoint (with awareness of replication lag for read-after-write consistency).

Q6: How would you implement observability for a microservices application?

Answer: I'd implement all three pillars — metrics, logs, and traces — with correlation between them, following the RED method (Rate, Errors, Duration) for each service.

Metrics (Prometheus + Grafana): Instrument every service with Prometheus client libraries. Key metrics per service: request rate (`http_requests_total` counter), error rate (5xx responses / total), latency (`http_request_duration_seconds` histogram with p50/p95/p99). Infrastructure metrics via node-exporter (CPU, memory, disk). Set up Grafana dashboards per service and a global overview dashboard. Configure Alertmanager rules: error rate > 1% for 5 minutes → PagerDuty, p99 latency > 500ms → Slack warning.

Structured Logging (Pino/Winston → Loki/ELK): JSON-formatted logs with consistent fields: `timestamp`, `level`, `service`, `traceId`, `spanId`, `requestId`, `event`, and contextual data. Never

log sensitive data (PII, tokens). Centralize via Fluentd/Fluent Bit DaemonSet collecting container logs. Query with LogQL (Loki) or KQL (CloudWatch Logs Insights).

Distributed Tracing (OpenTelemetry → Jaeger): Auto-instrument HTTP, gRPC, and database calls with OpenTelemetry SDK. Propagate W3C Trace Context headers across service boundaries. Every log entry includes `traceId` for correlation. Use traces to identify latency bottlenecks (which service/span is slow) and error propagation paths.

Correlation workflow: Alert fires (high error rate on order-service) → Check Grafana dashboard (spike in 500 errors) → Filter logs by traceId (error: "Payment gateway timeout") → View trace in Jaeger (payment-service → Stripe API took 30s, timed out) → Root cause: Stripe degradation.

Q7: Explain SLI, SLO, and error budgets with an example.

Answer: These three concepts form the foundation of Site Reliability Engineering, connecting reliability targets to engineering decisions.

SLI (Service Level Indicator) is a quantitative measure of service behavior. For an API: availability = successful requests / total requests; latency = proportion of requests faster than 300ms; error rate = failed requests / total requests. SLIs must be measurable from user perspective — measure at the load balancer, not inside the service.

SLO (Service Level Objective) is a target value for an SLI. "99.9% of requests will succeed (non-5xx) measured over a 30-day rolling window" or "99% of requests will complete in under 300ms." SLOs should be slightly tighter than SLAs (SLA is the contractual promise with consequences; SLO is the internal target with buffer).

Error Budget = 100% - SLO. For 99.9% availability SLO: error budget = 0.1% = 43.8 minutes of downtime per month. This budget represents how much unreliability is acceptable.

Error budget policy example:

- Budget > 50% remaining: Teams prioritize features, normal release cadence
- Budget 25-50%: Increase testing rigor, review risky changes
- Budget < 25%: Feature freeze, focus on reliability improvements
- Budget exhausted: All engineering effort on reliability until budget resets

This creates a data-driven balance between feature velocity and reliability. If we're within budget, we can push aggressive releases. If budget is burning fast (14.4x burn rate alert = budget exhausted in 2 hours), we pause feature work and investigate. This prevents both over-engineering reliability (100% uptime is infinitely expensive) and under-investing in reliability (constant outages destroy user trust).

Q8: How does AWS Lambda handle scaling and what are its limitations?

Answer: Lambda scales automatically and near-instantly by running each invocation in a separate execution environment (container). When concurrent requests increase, AWS creates new execution environments in parallel — from 0 to thousands of concurrent executions within seconds.

Scaling mechanism: Each Lambda function has a concurrency limit (1000 per region by default, soft limit). When a request arrives, Lambda either reuses a warm execution environment or creates a new one (cold start). Concurrency = number of simultaneous executions. Reserved concurrency guarantees capacity; provisioned concurrency pre-warms environments eliminating cold starts.

Cold start breakdown: Init duration (download code, start runtime: 100ms-2s depending on runtime and package size) + execution duration. Node.js/Python: ~100-300ms cold start. Java: ~1-3s. Mitigation: provisioned concurrency, ARM64 (Graviton2, ~20% faster), minimize package size, keep functions warm with scheduled pings (hacky).

Key limitations:

Limitation	Value	Workaround
Max timeout	15 minutes	Step Functions for longer workflows
Max memory	10,240 MB	More memory = proportionally more CPU
Max payload (sync)	6 MB	Store large data in S3, pass reference
Max deployment package	250 MB unzipped	Lambda layers, container images (10GB)
Max concurrent executions	1000 (default)	Request limit increase from AWS
/tmp storage	512 MB - 10 GB	S3 or EFS for larger temp storage
VPC cold start	+1-3s (ENI creation)	Hyperplane ENIs (improved), minimize VPC functions

Best practices for production: Keep functions focused (single responsibility, smaller packages). Use async invocation (SQS, EventBridge) for non-latency-sensitive work. Set appropriate memory (benchmark — 1024MB might be faster AND cheaper than 256MB due to proportional CPU).

Configure DLQ for failed invocations. Use Lambda Power Tuning tool to find optimal memory/cost ratio.

Q9: What is the difference between ConfigMaps and Secrets in Kubernetes?

Answer: Both inject configuration data into pods, but they differ in purpose, handling, and security.

ConfigMaps store non-sensitive configuration: environment variables, config files, command-line arguments. Data is stored as plain text (or plain base64 in etcd). Examples: log levels, feature flags, database hostnames, application properties files. ConfigMaps can be mounted as files (entire directory), injected as environment variables, or referenced in command arguments. Changes to ConfigMaps don't automatically restart pods — you need to restart or use a sidecar like Reloader to watch for changes.

Secrets store sensitive data: passwords, API keys, TLS certificates, database credentials. Data is base64 encoded (NOT encrypted by default — base64 is encoding, not encryption). Kubernetes supports encryption at rest via encryption providers configured on the API server. Secret types: Opaque (generic), kubernetes.io/tls, kubernetes.io/dockerconfigjson.

Key differences:

Aspect	ConfigMap	Secret
Purpose	Non-sensitive config	Sensitive credentials
Storage	Plain text in etcd	Base64 in etcd (encrypt at rest recommended)
Size limit	1 MB	1 MB
Volume mounting	Regular file	tmpfs (memory-backed, not written to disk)
RBAC	Standard	Should have restricted RBAC policies

Production best practices: Never store secrets in git or ConfigMaps. Use External Secrets Operator to sync from AWS Secrets Manager or HashiCorp Vault. Enable etcd encryption at rest. Restrict Secret access via RBAC (only necessary service accounts). Rotate secrets regularly with automated tooling. For 80 LPA interviews, emphasize that base64 in Secrets is NOT security — it's merely obfuscation.

Q10: How would you design a CI/CD pipeline for Kubernetes deployments?

Answer: I'd design a GitOps-based pipeline with automated testing gates, progressive deployment, and automatic rollback.

Pipeline stages:

1. **Trigger:** Push to feature branch → PR checks. Merge to main → full pipeline.
2. **Build:** Docker multi-stage build → push to ECR/GCR with SHA tag (`myapp:abc123`). Run Trivy/Snyk container scan. Fail on critical CVEs.
3. **Test:** Unit tests → integration tests (Testcontainers for DB/Redis) → contract tests (Pact for API compatibility). Minimum 80% coverage gate.
4. **Deploy to Staging:** Update Kubernetes manifest in Git repo (GitOps with ArgoCD/Flux). ArgoCD syncs staging cluster. Run smoke tests and E2E tests against staging.
5. **Deploy to Production (Progressive):**
 - Canary: Route 5% traffic to new version via Flagger/Istio. Monitor error rate and latency for 10 minutes.
 - If metrics healthy → 25% → 50% → 100%.
 - If error rate exceeds threshold → automatic rollback.
 - Blue/Green alternative: Deploy full green environment, switch traffic, keep blue for instant rollback.
6. **Post-deploy:** Verify SLO metrics, run synthetic monitoring checks.

GitOps structure:

```
k8s/
├── base/                # Kustomize base
│   ├── deployment.yaml
│   ├── service.yaml
│   └── kustomization.yaml
├── overlays/
│   ├── staging/         # staging-specific patches
│   └── production/     # production-specific patches
```

Rollback: `kubectl rollout undo` or revert Git commit (ArgoCD auto-syncs). With Flagger, automatic rollback on metric degradation.

Secrets management: Sealed Secrets or External Secrets Operator — never plain secrets in Git.

Q11: Explain Kubernetes networking – how does a packet reach a pod?

Answer: Kubernetes networking follows a flat model where every pod gets its own IP address, and pods can communicate directly without NAT. The path from external request to pod involves several components.

External → Pod path:

1. **External request** hits Cloud Load Balancer (AWS ALB/NLB, GCP LB) or Ingress controller (NGINX, Traefik).
2. **Ingress controller** (if using Ingress) reads Ingress rules, terminates TLS, and routes based on host/path to the appropriate Service ClusterIP.
3. **Service (ClusterIP):** kube-proxy on each node maintains iptables or IPVS rules. When a packet arrives for Service IP `10.96.0.5:80`, iptables DNAT (Destination NAT) rewrites the destination to a pod IP (e.g., `10.244.1.15:3000`) selected by load balancing algorithm (iptables random or IPVS round-robin).
4. **Pod network:** The packet arrives at the pod's network namespace via the CNI (Container Network Interface) plugin (Calico, Flannel, Cilium). Each pod has a veth pair connected to the node's network bridge or routed directly.

Pod → Pod (same node): Direct routing via bridge/CNI, no Service needed.

Pod → Pod (different nodes): CNI handles cross-node routing. Calico uses BGP for pod CIDR advertisement; Flannel uses VXLAN overlay network.

Pod → External: Source NAT (SNAT) on the node — external services see the node's IP, not the pod IP. Egress can be controlled via Network Policies.

DNS: CoreDNS resolves Service names (`api-service.default.svc.cluster.local`) to ClusterIP. Pod names in StatefulSets resolve directly to pod IPs via headless Services.

Network Policies: Firewall rules for pods. Default allow-all; policies can restrict ingress/egress by label, namespace, and port. Implemented by CNI plugin (Calico, Cilium).

Q12: How would you optimize AWS costs for a production application?

Answer: Cost optimization requires continuous effort across compute, storage, networking, and architectural decisions.

Compute:

- **Right-sizing:** Use AWS Compute Optimizer recommendations. Most EC2 instances are over-provisioned. Start small, scale up based on metrics.
- **Reserved Instances / Savings Plans:** 1-year or 3-year commitment for baseline capacity (30-72% savings). Use for stable workloads (RDS, baseline EC2).
- **Spot Instances:** Up to 90% discount for fault-tolerant workloads (Kubernetes node groups, CI/CD, batch processing). Use Spot Fleet with mixed On-Demand for stability.
- **Lambda:** Right-size memory (use Lambda Power Tuning). ARM64 for 20% savings. Avoid over-provisioning concurrency.

Storage:

- **S3 Intelligent-Tiering:** Auto-moves objects between access tiers. Lifecycle policies to Glacier for old data.
- **EBS:** Use gp3 (cheaper, configurable IOPS) over gp2. Delete unused volumes and snapshots.
- **RDS:** Aurora Serverless v2 for variable workloads. Reserved instances for production databases.

Networking:

- **CloudFront:** Cache static assets, reduce origin requests and data transfer costs.
- **VPC endpoints:** Avoid NAT Gateway charges (\$0.045/GB) for S3, DynamoDB access.
- **Same-AZ communication:** Cross-AZ data transfer costs \$0.01/GB each direction.

Architecture:

- **Serverless where appropriate:** API Gateway + Lambda for variable traffic (pay per request vs always-on EC2).
- **Caching:** ElastiCache reduces RDS load (smaller RDS instance needed).
- **SQS over always-on workers:** Scale to zero when no messages.

Monitoring: AWS Cost Explorer, set billing alerts, tag all resources (team, environment, project) for cost allocation. Review monthly, eliminate orphaned resources.

Q13: What is a Kubernetes Ingress and how does it differ from a LoadBalancer Service?

Answer: Both expose services externally, but they operate at different layers and serve different purposes.

A **LoadBalancer Service** creates a cloud provider load balancer (AWS ELB, GCP LB) for a single Kubernetes Service. It operates at Layer 4 (TCP/UDP) — routing based on IP and port. Each

LoadBalancer Service creates a separate cloud LB (~\$20/month each on AWS). If you have 10 services needing external access, you pay for 10 load balancers. Simple but expensive at scale.

An **Ingress** operates at Layer 7 (HTTP/HTTPS) and routes traffic to multiple Services based on host name and URL path — all through a single load balancer. One Ingress resource can route

`api.myapp.com/v1` → api-service, `api.myapp.com/v2` → api-v2-service, and `admin.myapp.com` → admin-service. This is cost-effective (one LB for many services) and feature-rich.

Ingress features beyond routing:

- TLS termination (HTTPS) with cert-manager for automatic Let's Encrypt certificates
- Path-based and host-based routing
- Rate limiting and request manipulation (via annotations)
- WebSocket support
- Authentication (OAuth2 proxy)

Ingress requires an Ingress Controller — a pod running NGINX, Traefik, or cloud-specific controller (AWS ALB Ingress Controller) that watches Ingress resources and configures the actual load balancer.

When to use what:

- LoadBalancer Service: Non-HTTP protocols (gRPC without Ingress support, TCP databases), single service needing external access, WebSocket before Ingress controller support
- Ingress: HTTP/HTTPS routing for multiple services, TLS termination, path-based routing, cost optimization

In production, the pattern is: Ingress Controller (one LB) → Ingress rules → Services → Pods.

Q14: How does ElastiCache Redis differ from using Redis in Kubernetes?

Answer: Both provide Redis caching, but they differ in operational model, reliability features, and integration.

AWS ElastiCache (managed):

- Fully managed — AWS handles patching, failover, backups, monitoring
- Multi-AZ with automatic failover (Redis Sentinel managed for you)
- Cluster mode for horizontal scaling (sharding across nodes)
- VPC integration, security groups, encryption at rest and in transit
- Backup and restore, snapshot scheduling

- CloudWatch metrics integration
- No ops overhead — just configure node type and replica count
- Cost: ~~\$0.017/hour for cache.r6g.large~~ (\$12/month) + data transfer
- Limitation: AWS-specific, less control over Redis configuration, version upgrades managed by AWS

Redis in Kubernetes (self-managed):

- Full control over Redis configuration, version, modules (RedisJSON, RedisSearch)
- Deploy via Helm chart (Bitnami Redis) or Redis Operator
- StatefulSet for stable identity and persistent storage
- Can run anywhere (multi-cloud, on-prem)
- Must manage: failover (Redis Sentinel or Cluster mode), backups, monitoring, patching
- Resource overhead: Redis pods consume node resources
- Cost: Node resources only (but includes operational cost of managing it)
- Dev/staging: Simple Redis Deployment with emptyDir (no persistence needed)

Decision matrix:

Factor	ElastiCache	K8s Redis
Ops burden	Zero	High
Production HA	Built-in Multi-AZ	Must configure Sentinel/Cluster
Cost at scale	Higher (\$)	Lower (node resources)
Control	Limited	Full
Multi-cloud	No	Yes
Best for	Production, team without Redis expertise	Dev/staging, multi-cloud, custom modules

For 80 LPA interviews: "I'd use ElastiCache for production — the operational overhead of managing Redis HA in K8s isn't worth the cost savings unless we're at massive scale or need specific Redis modules. For dev/staging, a simple Redis pod in K8s is fine."

Q15: How would you troubleshoot a pod stuck in CrashLoopBackOff?

Answer: CrashLoopBackOff means Kubernetes is repeatedly trying to start a container that keeps crashing. The backoff delay increases exponentially (10s, 20s, 40s... up to 5 minutes).

Systematic troubleshooting:

Step 1 — Check pod status and events:

```
kubectl describe pod <pod-name>
# Look at: State, Last State, Exit Code, Reason, Events section
# Exit Code 137 = OOMKilled (memory limit exceeded)
# Exit Code 1 = Application error
# Exit Code 143 = SIGTERM (graceful shutdown timeout)
```

Step 2 — Check container logs:

```
kubectl logs <pod-name>                # Current container logs
kubectl logs <pod-name> --previous        # Previous crashed container logs
kubectl logs <pod-name> -c <container>    # Specific container in multi-container
```

Step 3 — Common causes and fixes:

Cause	Symptoms	Fix
Application crash on startup	Exit code 1, error in logs	Fix application bug, missing env vars
OOMKilled	Exit code 137, Reason: OOMKilled	Increase memory limits, fix memory leak
Missing ConfigMap/Secret	CreateContainerConfigError	Create missing resource, check references
Failed health check	Container starts then restarts	Fix liveness probe, increase initialDelaySeconds
Image pull failure	ImagePullBackOff, ErrImagePull	Check image name/tag, imagePullSecrets
Permission denied	CrashLoopBackOff, permission errors	Run as non-root, check file permissions
Database not ready	Connection refused in logs	Add init container or startup probe

Step 4 — Debug interactively:

```
# Deploy debug container with shell access
kubectl debug <pod-name> -it --image=busybox --target=<container-name>

# Or override entrypoint temporarily
kubectl run debug --image=<same-image> --command -- sleep 3600
kubectl exec -it debug -- /bin/sh
# Manually run the application command to see errors
```

Step 5 — Check resource constraints:

```
kubectl top pod <pod-name> # Current resource usage
kubectl get limitrange -n <namespace> # Namespace defaults
kubectl get resourcequota -n <namespace> # Namespace quotas
```

Prevention: Implement proper health checks (startup probe for slow starts), set appropriate resource limits based on load testing, use init containers for dependency checks, and ensure ConfigMaps/Secrets exist before deployment.

9. Where to Practice

Kubernetes Hands-On

- [KillerCoda Kubernetes Playground](#) — Free K8s scenarios
- [Kubernetes Official Tutorials](#) — Official docs tutorials
- [KodeKloud CKA/CKAD Courses](#) — Certification prep with labs
- **Local setup:** minikube or kind (Kubernetes in Docker) for local practice

AWS Hands-On

- [AWS Free Tier](#) — 12 months free services
- [AWS Skill Builder](#) — Free digital courses
- [LocalStack](#) — Local AWS cloud emulator
- [AWS Well-Architected Labs](#) — Best practice labs

Observability

- [Prometheus + Grafana Docker Compose setup](#) — One-command monitoring stack
- [Jaeger Getting Started](#) — Tracing setup
- [Google SRE Book \(Free\)](#) — SLI/SLO/Error budgets

Certifications (Resume Boosters)

- **CKA** (Certified Kubernetes Administrator) — \$395, highly valued
- **CKAD** (Certified Kubernetes Application Developer) — For developers
- **AWS Solutions Architect Associate** — Foundational AWS cert
- **AWS DevOps Engineer Professional** — Advanced, for 80 LPA target

Interview Prep

- [Exponent — System Design + DevOps](#) — Mock interviews
- [ByteByteGo — System Design](#) — K8s and cloud chapters
- Design questions: "Design a CI/CD pipeline", "Design monitoring for microservices", "Design a multi-region deployment"

Key Topics to Drill

1. Write a Deployment + Service + Ingress YAML from memory
2. Explain pod scheduling, HPA, and Cluster Autoscaler interaction
3. Design observability stack for 10 microservices
4. Calculate error budget and burn rate for 99.9% SLO
5. Troubleshoot CrashLoopBackOff, OOMKilled, and network policy issues

Last updated: August 2026 | Target: 80+ LPA SDE (Google L5, Meta E4, Amazon SDE3, Microsoft L63+)

Kubernetes, Docker, Cloud, and Observability Interview – 50 Questions

MERN MASTERY · 1 CR PRODUCT PREP · HARSH VARDHAN GUPTA

This bank covers Docker, Kubernetes (K8s), Amazon Web Services (AWS) basics, and observability for product company interviews. Spell out abbreviations on first use.

1. What is Docker and why is it used?

Docker is a platform for building, shipping, and running applications in lightweight, portable containers. Containers package application code with all dependencies (libraries, runtime) ensuring consistent behavior across development, testing, and production. Docker solves "it works on my machine" problems and enables microservices deployment at scale.

2. What is the difference between a container and a virtual machine (VM)?

A Virtual Machine (VM) runs a full guest operating system on a hypervisor—heavy (gigabytes), slow to start (minutes). A container shares the host operating system kernel, isolating only the application and its dependencies—lightweight (megabytes), fast to start (seconds). Containers are ideal for microservices; VMs for running different operating systems or strong isolation requirements.

3. What is a Docker image vs a container?

A Docker image is an immutable template (blueprint) containing application code, runtime, libraries, and configuration—stored in layers for efficient caching. A container is a running instance of an image—mutable, with its own filesystem, network, and process space. Analogy: image is a class, container is an object. You can run multiple containers from one image.

4. What is a Dockerfile?

A Dockerfile is a text file with instructions to build a Docker image: **FROM** (base image), **COPY** (add files), **RUN** (execute commands), **EXPOSE** (port), **CMD** (default command). Example: **FROM**

`node:20` , `COPY . /app` , `RUN npm install` , `CMD ["node", "server.js"]` . Build with `docker build -t myapp .` . Best practices: multi-stage builds, non-root user, minimal base images.

5. What is Docker Compose?

Docker Compose defines and runs multi-container applications using a YAML (YAML Ain't Markup Language) file. Specify services (web, database, cache), networks, and volumes in `docker-compose.yml` . Run with `docker compose up` . Ideal for local development environments replicating production topology. Not for production orchestration—use Kubernetes instead.

6. What is Kubernetes (K8s)?

Kubernetes (K8s—K plus 8 letters plus s) is an open-source container orchestration platform that automates deployment, scaling, and management of containerized applications. Originally from Google, now maintained by Cloud Native Computing Foundation (CNCF). Handles scheduling containers across nodes, self-healing (restart failed containers), rolling updates, and service discovery. Industry standard for production container workloads.

7. What is a Kubernetes pod?

A pod is the smallest deployable unit in Kubernetes—one or more containers sharing network and storage. Containers in a pod share localhost and volumes. Usually one container per pod (one process per pod principle). Pods are ephemeral—Kubernetes creates new pods when old ones die. Deployments manage pod lifecycle.

8. What is a Kubernetes Deployment?

A Deployment manages a set of identical pods, handling rolling updates, rollbacks, and scaling. Define desired replica count and container spec in YAML. Kubernetes continuously reconciles actual state to desired state. `kubectl apply -f deployment.yaml` creates or updates. Use Deployments for stateless applications (web servers, APIs).

9. What is a Kubernetes Service?

A Service provides a stable network endpoint (IP address and DNS name) for a set of pods. Types: ClusterIP (internal only), NodePort (expose on node port), LoadBalancer (cloud provider load

balancer), ExternalName. Pods come and go; Services route traffic to healthy pods via label selectors. Essential because pod IPs change on restart.

10. What is an Ingress in Kubernetes?

An Ingress manages external HTTP/HTTPS (Hypertext Transfer Protocol Secure) access to cluster services. Defines routing rules (host, path) to backend services. Requires an Ingress controller (Nginx, Traefik, AWS Load Balancer Controller). Handles SSL termination, path-based routing, and virtual hosts. One Ingress replaces multiple LoadBalancer services for cost efficiency.

11. What is a ConfigMap vs a Secret in Kubernetes?

ConfigMap stores non-sensitive configuration (environment variables, config files) as key-value pairs—database URLs, feature flags. Secret stores sensitive data (passwords, API keys, TLS certificates)—base64 encoded by default, not encrypted (enable encryption at rest). Mount both as environment variables or files in pods. Never commit secrets to git—use external secret managers.

12. What is a PersistentVolume (PV) and PersistentVolumeClaim (PVC)?

PersistentVolume (PV) is cluster-level storage resource (like a disk) provisioned by admin or dynamically. PersistentVolumeClaim (PVC) is a pod's request for storage (size, access mode). Kubernetes binds PVC to available PV. Data persists beyond pod lifecycle—essential for databases and stateful applications. Storage classes enable dynamic provisioning on AWS Elastic Block Store (EBS), Google Persistent Disk.

13. What is kubectl?

kubectl is the command-line tool for interacting with Kubernetes clusters. Common commands: `kubectl get pods`, `kubectl describe pod <name>`, `kubectl logs <pod>`, `kubectl apply -f <file>`, `kubectl scale deployment <name> --replicas=5`, `kubectl exec -it <pod> -- /bin/sh`. Essential for debugging, deployment, and operations.

14. What is a Kubernetes namespace?

A namespace is a virtual cluster within a physical cluster for resource isolation. Default namespaces: `default`, `kube-system`, `kube-public`. Teams use namespaces to separate environments (dev, staging, prod) or teams. Resource quotas and network policies can be scoped per namespace.

```
kubectl get pods -n production .
```

15. What is Horizontal Pod Autoscaler (HPA)?

Horizontal Pod Autoscaler (HPA) automatically scales the number of pod replicas based on metrics—typically CPU (Central Processing Unit) utilization or custom metrics (requests per second). Define min/max replicas and target utilization. HPA adds pods when load increases, removes when load decreases. Requires metrics server installed. Pair with Cluster Autoscaler for node-level scaling.

16. What is liveness vs readiness probes?

Liveness probe checks if a container is alive—if it fails, Kubernetes restarts the container (fix deadlocks). Readiness probe checks if a container is ready to serve traffic—if it fails, Kubernetes removes it from Service endpoints (handle startup time, temporary overload). Configure both in pod spec with HTTP, TCP (Transmission Control Protocol), or exec checks.

17. What is a rolling update in Kubernetes?

Rolling update gradually replaces old pods with new ones during a deployment—zero downtime. Strategy: `maxUnavailable` (how many can be down) and `maxSurge` (how many extra during update). If new pods fail readiness checks, rollout pauses automatically. Rollback with `kubectl rollout undo`. Default deployment strategy for production releases.

18. What is Helm?

Helm is the package manager for Kubernetes—charts are templated YAML bundles for deploying applications. Install complex stacks (Prometheus, PostgreSQL) with `helm install`. Supports versioning, rollback, and value overrides. Helm charts reduce boilerplate and standardize deployments across environments. CNCF graduated project.

19. What is Amazon Web Services (AWS)?

Amazon Web Services (AWS) is the leading cloud computing platform offering on-demand infrastructure: compute, storage, databases, networking, and managed services. Pay-as-you-go pricing across global regions and availability zones. Core services: EC2 (compute), S3 (storage), RDS (database), Lambda (serverless), EKS (Kubernetes). Most product companies use AWS, Google Cloud Platform (GCP), or Microsoft Azure.

20. What is EC2 (Elastic Compute Cloud)?

Elastic Compute Cloud (EC2) provides resizable virtual servers (instances) in the cloud. Choose instance types (CPU, memory, GPU), operating system, and storage. Auto Scaling Groups adjust capacity based on demand. Security Groups act as virtual firewalls. EC2 is the foundation for running applications before containers—or as Kubernetes worker nodes.

21. What is S3 (Simple Storage Service)?

Simple Storage Service (S3) is object storage for any amount of data—files, images, backups, logs. Buckets store objects keyed by name. Features: versioning, lifecycle policies (archive to Glacier), pre-signed URLs for direct upload, Static Website Hosting. 99.999999999% (11 nines) durability. Used for media storage, data lakes, and static asset hosting.

22. What is RDS (Relational Database Service)?

Relational Database Service (RDS) is managed SQL databases—PostgreSQL, MySQL, Oracle, SQL Server. AWS handles backups, patching, replication, and failover. Multi-AZ (Availability Zone) deployment for high availability. Read replicas for read scaling. Use RDS when you need managed operations; self-manage on EC2 for full control.

23. What is AWS Lambda?

AWS Lambda runs code without managing servers—serverless compute. Upload function, trigger on events (HTTP via API Gateway, S3 upload, Kafka message). Pay per invocation and duration (millisecond billing). Auto-scales from zero to thousands. Ideal for event processing, API backends with variable traffic, and scheduled tasks. Cold start latency is a consideration.

24. What is Amazon Elastic Container Registry (ECR)?

Elastic Container Registry (ECR) is AWS's managed Docker container registry. Store, manage, and deploy container images. Integrates with EKS and ECS (Elastic Container Service). Images scanned for vulnerabilities. Push with `docker push`; Kubernetes pulls via IAM role. Private registry—no need for Docker Hub in AWS deployments.

25. What is EKS (Elastic Kubernetes Service)?

Elastic Kubernetes Service (EKS) is AWS's managed Kubernetes. AWS manages the control plane (API server, etcd, scheduler); you manage worker nodes (EC2 or Fargate). Integrates with AWS VPC, IAM, ECR, and CloudWatch. Production-ready Kubernetes without managing control plane availability. Use for containerized microservices at scale on AWS.

26. What is a Virtual Private Cloud (VPC)?

Virtual Private Cloud (VPC) is an isolated network within AWS where you launch resources. Define IP ranges (CIDR blocks), subnets (public for load balancers, private for apps/databases), route tables, and security groups. VPC peering and VPN connect to other networks. Every AWS resource runs inside a VPC—network security foundation.

27. What is IAM (Identity and Access Management)?

Identity and Access Management (IAM) controls who can access AWS resources and how. Users, groups, roles, and policies (JSON documents defining permissions). Principle of least privilege: grant minimum required permissions. Roles (not access keys) for EC2 instances and Lambda functions. Enable Multi-Factor Authentication (MFA) for root account. Audit with AWS CloudTrail.

28. What is CloudWatch?

Amazon CloudWatch monitors AWS resources and applications. Collect metrics (CPU, memory, request count), logs (centralized log groups), and alarms (notify on thresholds). Dashboards visualize system health. CloudWatch Logs Insights queries log data. Integrates with SNS (Simple Notification Service) for alerts. Foundation of AWS observability—pair with X-Ray for tracing.

29. What is observability?

Observability is the ability to understand a system's internal state from its external outputs—logs, metrics, and traces. More than monitoring (known unknowns); observability handles unknown unknowns (unexpected failures). Three pillars: logs (what happened), metrics (how much/how fast), traces (request flow across services). Essential for debugging distributed microservices.

30. What is the difference between monitoring and observability?

Monitoring checks known metrics against thresholds (CPU > 80% → alert)—reactive, predefined. Observability enables exploratory analysis of any system behavior through rich telemetry—proactive, ad-hoc queries. Monitoring tells you something is wrong; observability helps you understand why. Both are needed: monitoring for alerts, observability for root cause analysis.

31. What is Prometheus?

Prometheus is an open-source monitoring system that scrapes metrics from targets at intervals, stores time-series data, and evaluates alerting rules. Uses PromQL (Prometheus Query Language) for queries. Pull-based model with service discovery. Standard for Kubernetes monitoring—kubernetes-stack deploys Prometheus, Grafana, and Alertmanager together.

32. What is Grafana?

Grafana is an open-source visualization platform for metrics, logs, and traces. Connects to Prometheus, Elasticsearch, CloudWatch, and 100+ data sources. Create dashboards with graphs, gauges, and heatmaps. Alerting rules notify via Slack, PagerDuty, email. Industry standard for operational dashboards—every DevOps team uses Grafana.

33. What is distributed tracing?

Distributed tracing tracks a single request as it flows through multiple microservices, showing latency at each step. Each service creates spans linked by trace ID. Identifies bottlenecks (which service is slow?) and failure points. Tools: Jaeger, Zipkin, AWS X-Ray, OpenTelemetry. Critical when a user request touches 10+ services.

34. What is OpenTelemetry?

OpenTelemetry (OTel) is a vendor-neutral standard for generating, collecting, and exporting telemetry data (traces, metrics, logs). Provides SDKs (Software Development Kits) for all languages and a collector for processing. Replaces vendor-specific agents (Datadog, New Relic proprietary SDKs). CNCF project—future-proof your observability instrumentation.

35. What is the ELK stack?

ELK stack: Elasticsearch (search and analytics engine), Logstash (log processing pipeline), Kibana (visualization dashboard). Collects, processes, and visualizes logs centrally. Modern variant: EFK (Elasticsearch, Fluentd, Kibana)—Fluentd replaces Logstash as lighter log shipper. Essential for searching production logs across hundreds of microservices.

36. What is a Service Level Objective (SLO) in observability?

Service Level Objective (SLO) is a target reliability metric—e.g., 99.9% of requests complete under 200 milliseconds over 30 days. Service Level Indicator (SLI) measures actual performance; error budget is allowed downtime ($0.1\% = \sim 43$ minutes/month). SLOs drive engineering priorities: if error budget is exhausted, focus on reliability over features.

37. What is a runbook?

A runbook is a documented procedure for handling operational scenarios—production outage, database failover, scaling event. Contains: symptoms, diagnosis steps, remediation commands, escalation paths. On-call engineers follow runbooks during incidents. Good runbooks reduce Mean Time To Recovery (MTTR). Store in wiki or tools like PagerDuty or Grafana OnCall.

38. What is Infrastructure as Code (IaC)?

Infrastructure as Code (IaC) manages infrastructure through machine-readable definition files rather than manual configuration. Version-controlled, reviewable, repeatable. Tools: Terraform (multi-cloud), AWS CloudFormation (AWS-native), Pulumi (programming languages). Benefits: reproducible environments, disaster recovery, and audit trail. Essential for DevOps maturity.

39. What is Terraform?

Terraform by HashiCorp defines infrastructure using HashiCorp Configuration Language (HCL) files. Plan before apply (preview changes). State file tracks current infrastructure. Supports AWS, GCP, Azure, Kubernetes. Modules reuse configurations. `terraform plan` → `terraform apply`. Industry standard for Infrastructure as Code across cloud providers.

40. What is GitOps?

GitOps uses git as the single source of truth for infrastructure and application deployment. Changes go through pull requests; automated pipelines sync cluster state to git. Tools: ArgoCD, Flux for Kubernetes. Benefits: audit trail, rollback via git revert, and consistent deployments. Extends Infrastructure as Code principles to runtime operations.

41. What is a sidecar container pattern?

Sidecar pattern runs a helper container alongside the main application container in the same pod. Sidecar handles cross-cutting concerns: logging (Fluentd ships logs), proxy (Envoy for service mesh), or monitoring (metrics exporter). Main container stays focused on business logic. Common in Istio service mesh and log collection.

42. What is AWS Fargate?

AWS Fargate runs containers without managing servers—serverless compute for containers. No EC2 instances to provision or patch. Specify CPU and memory; Fargate handles infrastructure. Works with ECS and EKS. Good for variable workloads and teams without dedicated infrastructure engineers. Trade-off: less control, potentially higher cost at steady high load.

43. What is a stateful vs stateless workload in Kubernetes?

Stateless workloads (web APIs, workers) store no data in the container—any pod can handle any request. Use Deployments. Stateful workloads (databases, Kafka brokers) need stable network identity and persistent storage. Use StatefulSets with ordered pod names and PersistentVolumeClaims. Never run production databases as plain Deployments.

44. What is a StatefulSet in Kubernetes?

StatefulSet manages stateful applications with guaranteed pod identity and stable storage. Pods get predictable names (`web-0` , `web-1`), ordered deployment and scaling, and persistent volume per pod. Used for databases (PostgreSQL, MongoDB), Kafka brokers, and ZooKeeper. Headless Service provides stable DNS for each pod.

45. What is network policy in Kubernetes?

NetworkPolicy controls traffic between pods—allow/deny rules based on labels, namespaces, and ports. Default: all pods can communicate (open). Restrict: only frontend pods can reach backend pods on port 8080. Implements micro-segmentation for defense in depth. Requires a network plugin supporting NetworkPolicy (Calico, Cilium).

46. What is RBAC in Kubernetes?

Role-Based Access Control (RBAC) restricts who can perform which actions in a Kubernetes cluster. Roles define permissions (get pods, create deployments); RoleBindings assign roles to users/groups/service accounts. ClusterRoles apply cluster-wide. Principle of least privilege: developers get namespace-scoped access; CI/CD (Continuous Integration/Continuous Deployment) gets deployment permissions only.

47. What is pod resource requests and limits?

Requests: guaranteed CPU and memory allocated to a pod—scheduler uses for placement. Limits: maximum CPU and memory a pod can consume—exceeding memory limit kills the pod (Out Of Memory—OOM). Set both for production: requests ensure scheduling; limits prevent resource hogging. Right-size based on actual usage metrics from Prometheus.

48. What is cluster autoscaler?

Cluster Autoscaler automatically adjusts the number of nodes in a Kubernetes cluster based on pending pods. If pods cannot be scheduled (insufficient resources), it adds nodes. If nodes are underutilized, it removes them. Works with cloud provider auto scaling groups (AWS Auto Scaling Groups). Pair with HPA for full automatic scaling.

49. How do you debug a crashing pod in Kubernetes?

Steps: `kubectl get pods` (check status: CrashLoopBackOff), `kubectl describe pod <name>` (events, exit codes), `kubectl logs <pod>` (application logs), `kubectl logs <pod> --previous` (logs from crashed container), `kubectl exec -it <pod> -- /bin/sh` (interactive debug if running). Common causes: OOM, missing environment variables, failed liveness probe, image pull errors.

50. Summarize Kubernetes and cloud interview checklist?

Cover: Docker fundamentals (image, container, Dockerfile), Kubernetes core objects (Pod, Deployment, Service, Ingress), scaling (HPA, cluster autoscaler), AWS basics (EC2, S3, RDS, Lambda, EKS, VPC, IAM), observability (Prometheus, Grafana, ELK, tracing, SLOs), Infrastructure as Code (Terraform, GitOps), and debugging workflow. Draw architecture diagrams showing how components connect in production.